DEVOPS COURSE · SESSION 21 CAPSTONE

PantryPal

A pantry expiry tracker taken through the whole DevOps lifecycle: code, tests, containers, CI/CD, security scanning, Terraform, Kubernetes and monitoring.

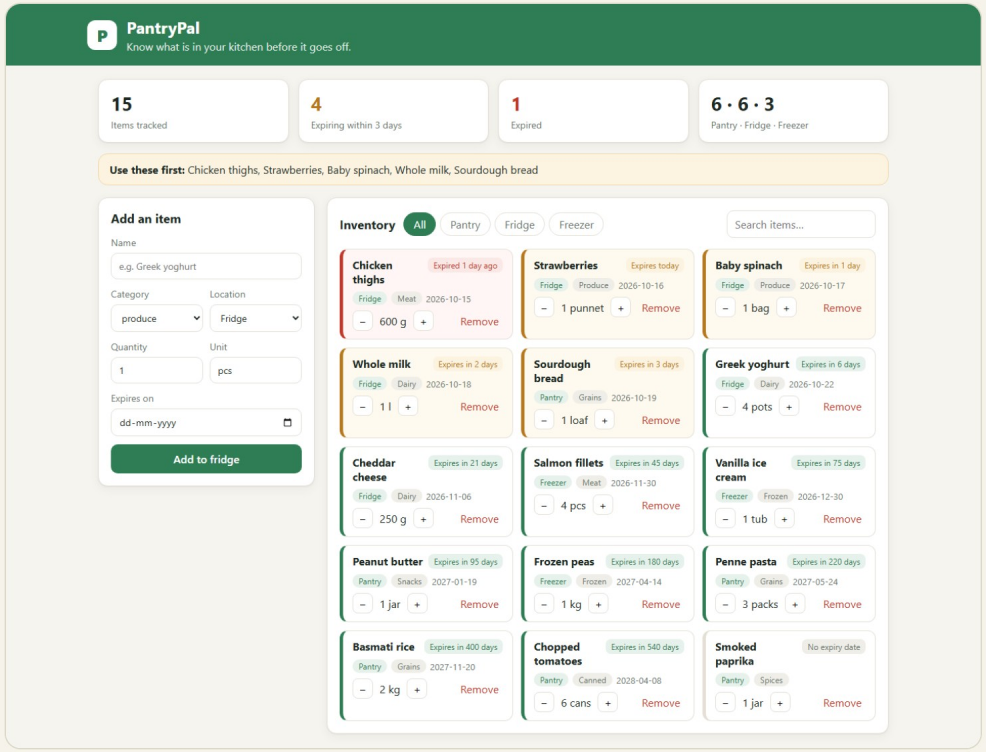
`github.com/ARRY7686/pantrypal`

FastAPI · React · PostgreSQL · Docker · GitHub Actions · Trivy · Terraform · Helm · Prometheus · Grafana

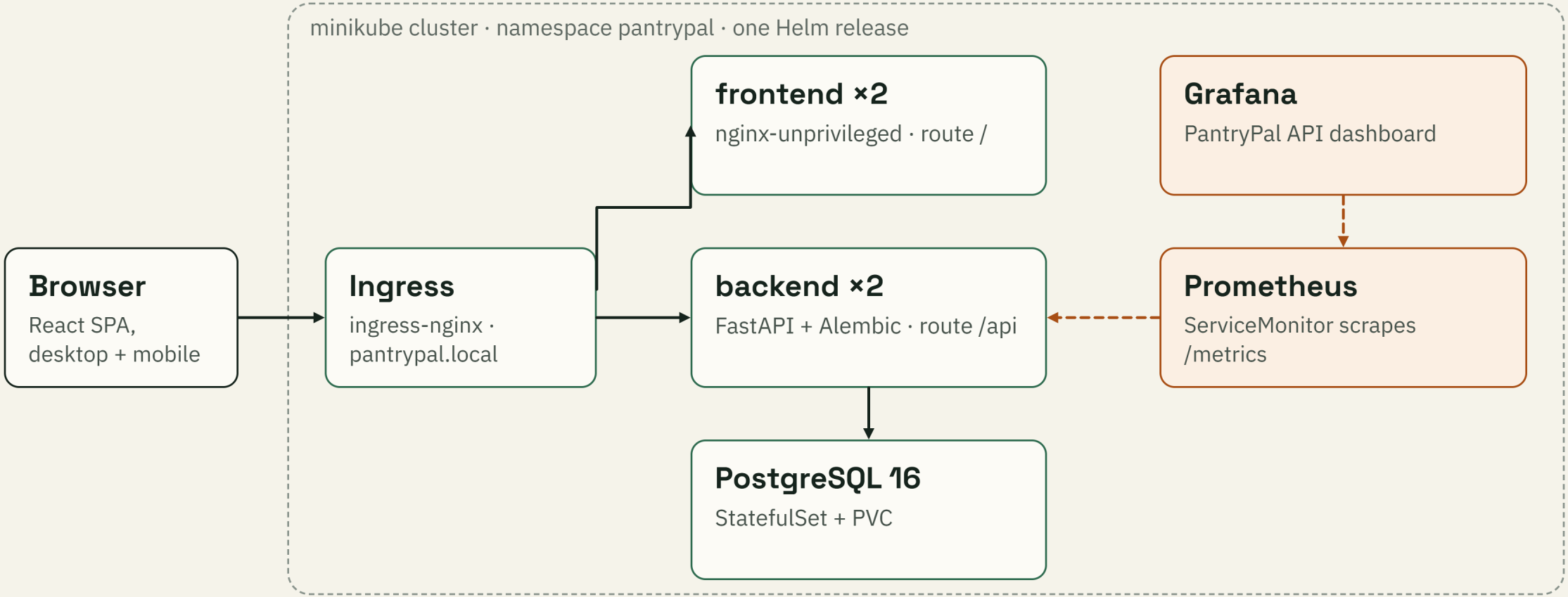
What PantryPal does

Know what's in your kitchen before it expires.

- Add items with quantity and expiry date
- Filter by pantry, fridge or freezer; search by name
- Use items up with +/− buttons
- Expiring-soon and expired warnings
- Out-of-stock badge and per-location counts



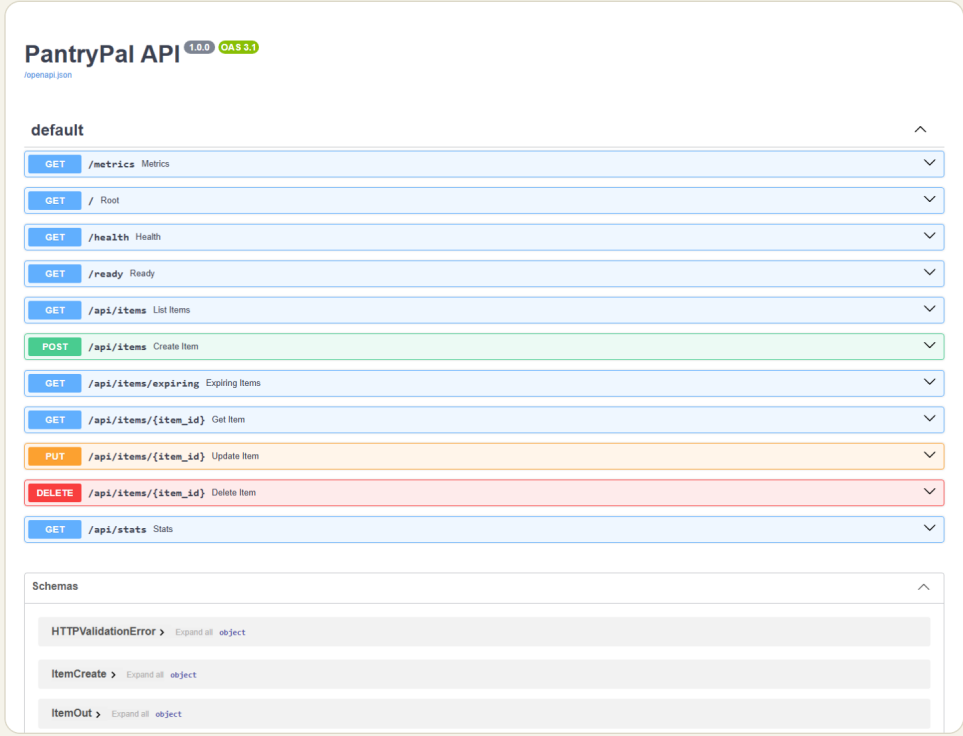
How the pieces fit



FastAPI, PostgreSQL and Alembic

Method	Path	Purpose
GET	/health	Liveness probe
GET	/ready	Readiness: database reachable
GET	/metrics	Prometheus metrics
GET, POST	/api/items	List (?location, ?q), create
GET, PUT, DELETE	/api/items/{id}	Read, update, delete
GET	/api/items/expiring	Expiring soon and expired
GET	/api/stats	Counts per location

Schema: one **items** table, created by Alembic migration 0001. Migrations take a PostgreSQL advisory lock, so two backend replicas starting together never race.



04 · TESTING

Tests gate every build

12

pytest tests passed, covering every endpoint

99%

line coverage of the backend

0

ruff lint and format findings

Each test runs against its own SQLite database from a **conftest.py** fixture, so tests never touch PostgreSQL or each other. In CI the pytest step comes first: if it fails, no image is built.

05 · DOCKER

Non-root images, health-gated startup

backend

`python:3.12-slim`

- Two stages: dependencies built apart, copied in
- Runs as uid 10001
- Runs Alembic migrations, then uvicorn on :8000
- HEALTHCHECK on /health

frontend

`node:22 → nginx`

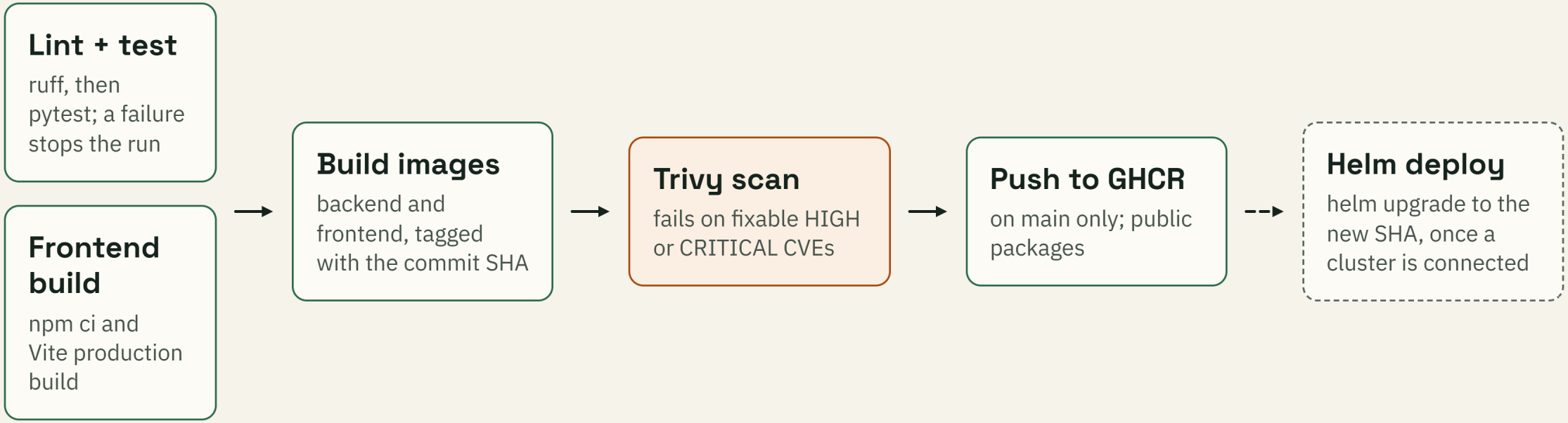
- Node stage builds the Vite bundle
- nginx-unprivileged serves it as uid 101 on :8080
- Backend address set by BACKEND_URL at start-up

postgres

`postgres:16-alpine`

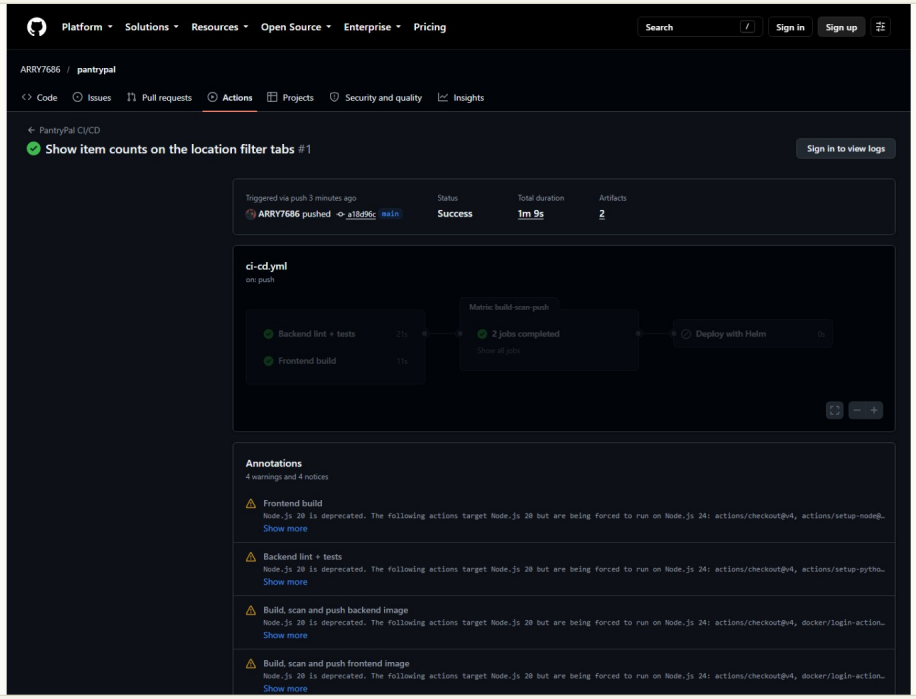
- Named volume keeps the data
- pg_isready healthcheck
- Compose starts each service only after the one before it is healthy

One GitHub Actions workflow, push to deploy

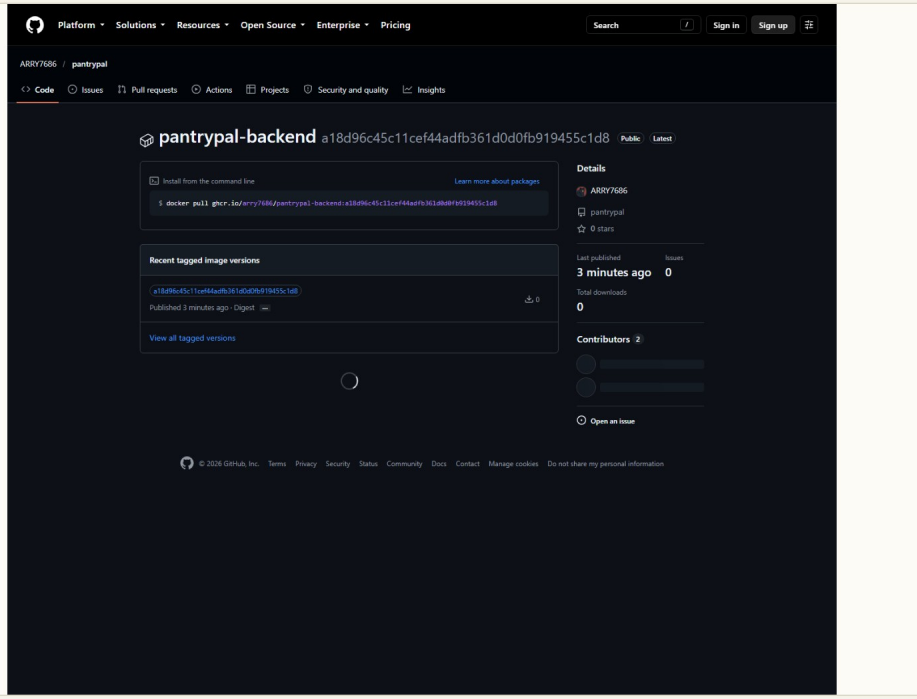


Images are never tagged **latest**: every tag is a commit SHA, so each deployment traces back to one commit. The workflow was also run locally with **act** before the first push.

Green on GitHub, images on GHCR



Actions run: tests, frontend build, both image builds, Trivy scans and pushes succeeded; deploy skipped



GHCR: ghcr.io/arry7686/pantrypal-backend and -frontend, tagged with the full commit SHA

Trivy failed the first build, and it was right

3 HIGH CVEs

found in starlette 0.46.2, pulled in by FastAPI 0.115

Upgrade

FastAPI 0.143.2 with Starlette 1.8.0; tests still green

0 fixable

HIGH or CRITICAL findings left in either image; the gate passes

The gate uses **--severity HIGH,CRITICAL --ignore-unfixed --exit-code 1**. CVEs in the Debian base with no released fix are still reported but do not block, since there is nothing to upgrade to yet.

08 · TERRAFORM

AWS VPC and EKS as code

52
to add in terraform plan

52
added by terraform apply

52
removed by terraform destroy

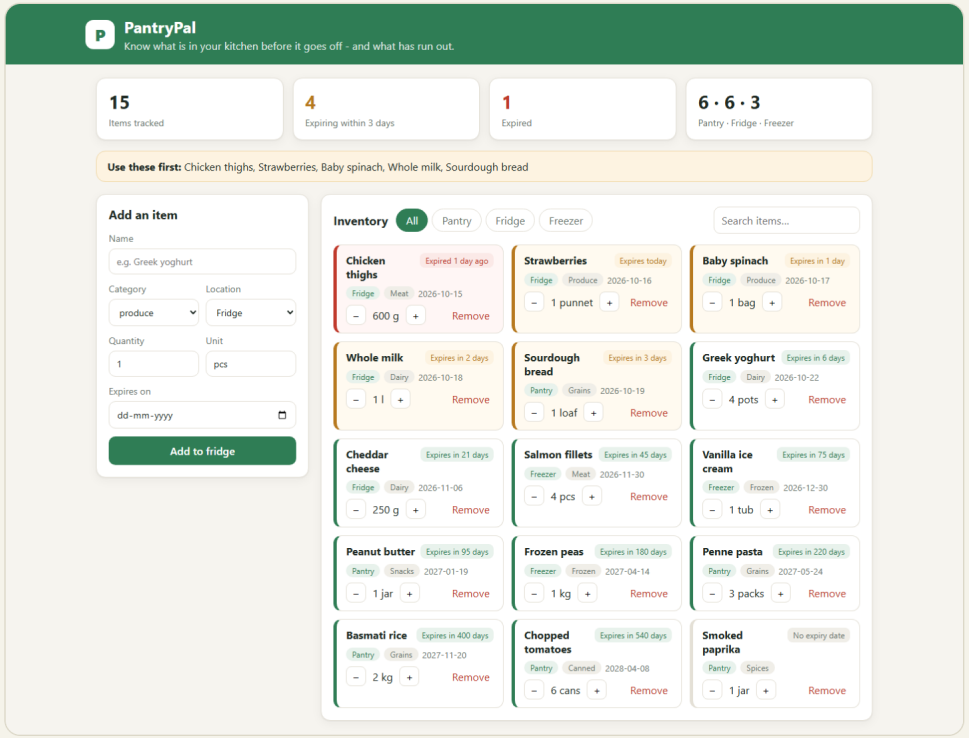
- VPC 10.30.0.0/16: 2 public + 2 private subnets, NAT gateway
- EKS 1.31 cluster pantrypal-dev-eks in ap-south-1
- Managed node group: t3.medium, min 1, desired 2, max 3

There was no AWS account, so apply and destroy ran on **Moto**, a local AWS emulator. Only environment variables differ: with real credentials the same commands build the real cluster.

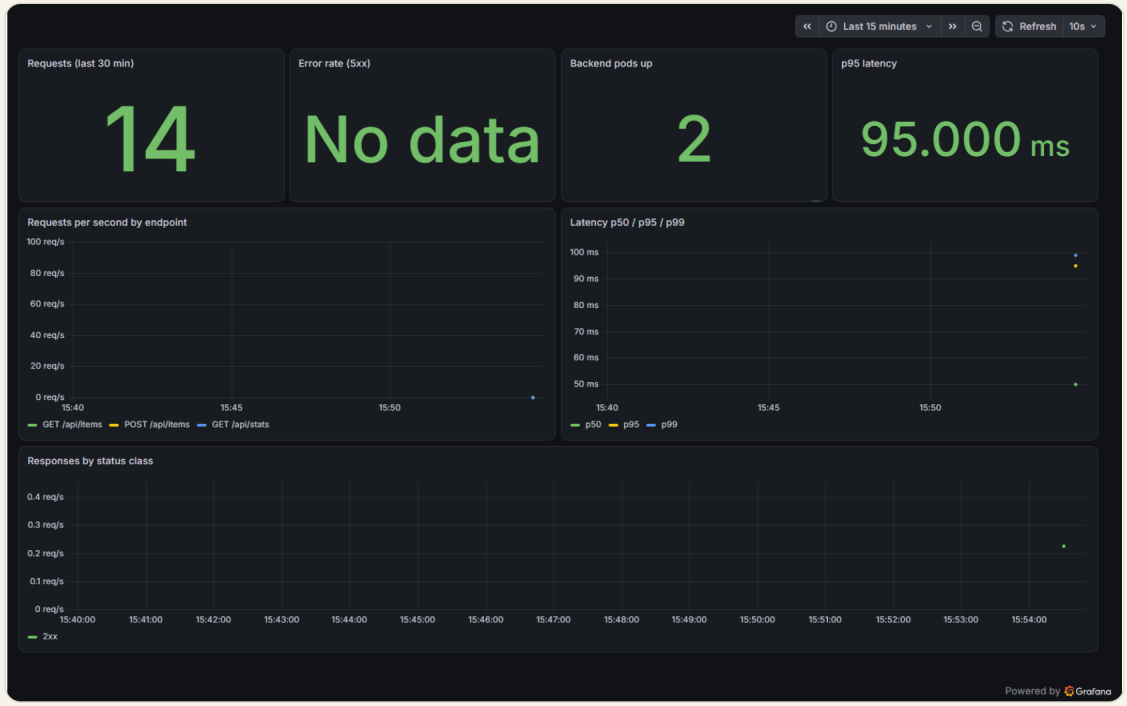
One Helm chart runs the whole app

- Namespace pantrypal from k8s/namespace.yaml
- backend ×2 and frontend ×2 behind ClusterIP Services
- PostgreSQL StatefulSet with a persistent volume
- Ingress pantrypal.local: / to the frontend, /api to the backend
- Readiness on /ready, liveness on /health
- Read-only root filesystem, all Linux capabilities dropped

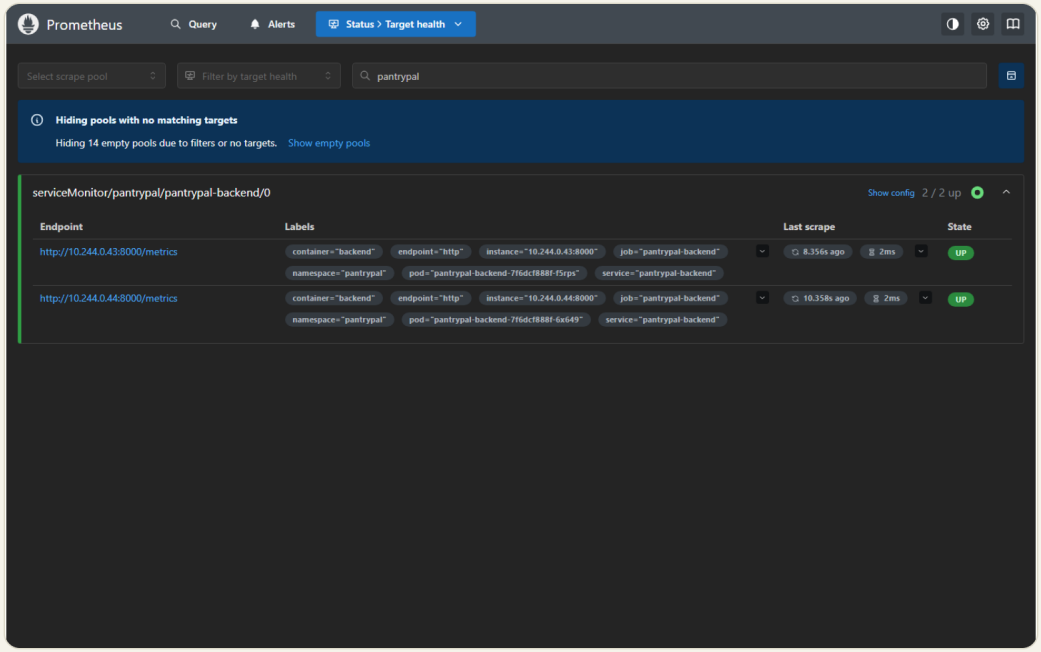
On minikube: STATUS deployed, all 5 pods Running, app served through the Ingress.



Prometheus scrapes it, Grafana shows it



Grafana: dashboard shipped by the chart. Requests per endpoint, 2 pods up, p95 latency about 95 ms.



Prometheus: both backend pods up, found through a ServiceMonitor; the backend exposes /metrics.

From commit to running pods

1 **Commit a change**

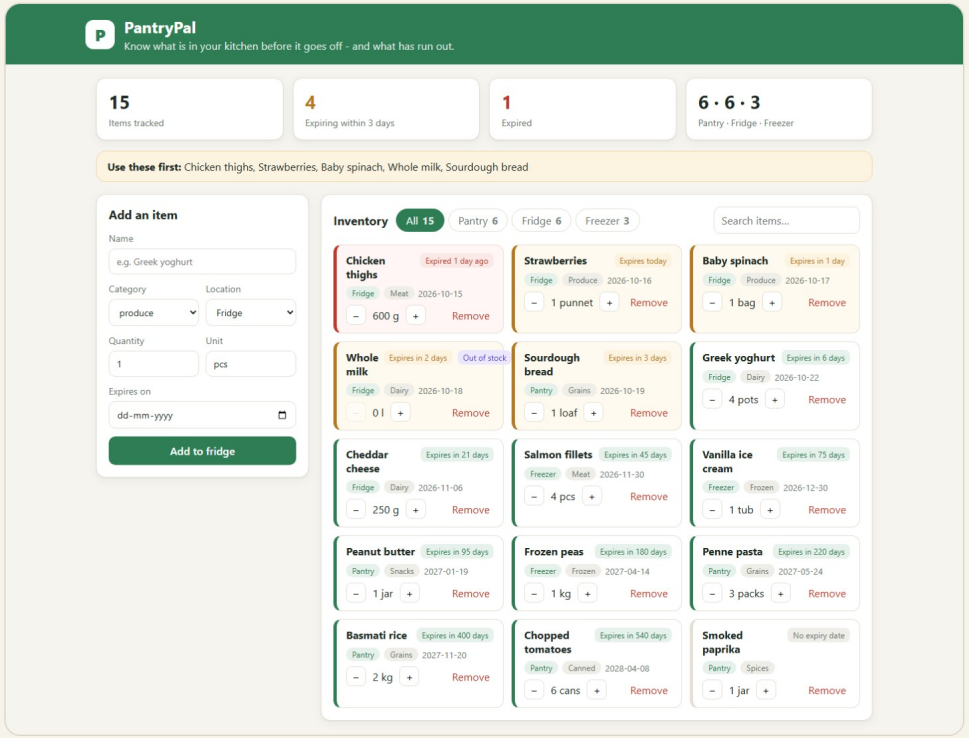
“Show item counts on the location filter tabs”, pushed to main

2 **Pipeline runs**

Tests pass, both images build clean under Trivy and land on GHCR as tag a18d96c

3 **Deployment updates**

helm upgrade to tag a18d96c gives Helm revision 2; the browser shows the new tab counts



SUMMARY

All ten modules, with evidence

Module	What was built	Result
M1 Application	React + FastAPI + PostgreSQL, Alembic migration	Runs with docker compose up
M2 Testing	12 pytest tests on an isolated SQLite DB	12 passed, 99% coverage
M3 Git	Public repo, 19 commits, .gitignore	gitleaks: no leaks
M4 Docker	Multi-stage, non-root; health-gated compose	All 3 services healthy
M5 CI/CD	Actions: tests → build → Trivy → GHCR	Green runs, images tagged by SHA
M6 DevSecOps	Trivy gate on both images	3 HIGH CVEs caught and fixed
M7 Terraform	VPC + EKS 1.31 + managed node group	Plan, apply, destroy: 52 each
M8 Kubernetes	Namespace, Helm chart, 2+2 replicas, Ingress	All pods Running on minikube
M9 Observability	/metrics, Prometheus, Grafana dashboard	Targets up, panels live
M10 Docs + demo	README with evidence; commit-to-deploy demo	Helm revision 2 on the new SHA

HONEST LIMITS

What is not proven yet, and what comes next

Limits

- Terraform ran on the Moto emulator, so there is no AWS Console screenshot
- The deploy job is skipped: no real cluster is connected to GitHub
- Actions warns that some actions still use the Node 20 runtime

Next steps

- Apply the same Terraform to a real AWS account
- Set DEPLOY_ENABLED and KUBE_CONFIG so every push deploys
- Turn on the HPA and add Prometheus alert rules

Thank you

Code, screenshots and terminal output for every module:

github.com/ARRY7686/pantrypal